

DevSecOps Automated Application Security Pipeline - Project Report

1. Introduction

This project demonstrates a shift-left DevSecOps approach to prevent insecure code from entering the repository. Security checks are performed before commit and during Continuous Integration to ensure only secure code is accepted.

2. Architecture Workflow

Developer writes code -> Pre-commit security hooks -> Git push -> GitHub Actions security scan -> Secure code allowed

3. Tools Used

- Flask – Demo web application
- TruffleHog – Secret detection
- Semgrep – Static Application Security Testing (SAST)

4. Week 1 – Secret Detection (TruffleHog)

A pre-commit hook was configured to scan source code using TruffleHog before committing changes.

If any API key or credential is detected, the commit is automatically blocked.

This prevents secrets from entering Git history.

```
(venv) (base) bharathreddy@Bharaths-MacBook-Pro Project_1 % rm .sengrep.yml
(venv) (base) bharathreddy@Bharaths-MacBook-Pro Project_1 % nano .pre-commit-config.yaml
(venv) (base) bharathreddy@Bharaths-MacBook-Pro Project_1 % pre-commit clean
Cleaned /Users/bharathreddy/.cache/pre-commit.
(venv) (base) bharathreddy@Bharaths-MacBook-Pro Project_1 % pre-commit run --all-files
trufflehog.....
- hook id: trufflehog
- exit code: 1

=====
Reason: High Entropy
Date: 2026-02-21 00:18:53
Hash: 242f27748342eca38f70b74718293343463cfa98
Filepath: app/gd.py
Branch: origin/main
Commit: Testing stripe secret block for presentation

@@ -1,10 +1,18 @@
-# A simple, secure script to test secret detection
+
+def run_application():
+    print('Application is starting up securely...')
+
+--- GUARANTEED TRUFFLEHOG BLOCK ---
+
+--- We know this exact key triggers the Stripe detector.
+--- Because there is no web code here, Sengrep will pass it cleanly.
+API_KEY = "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"
+--- STRUCTURAL SECRET (GUARANTEED BLOCK FOR SCREENSHOT) ---
+
+--- Sengrep will ignore this (not a code vulnerability)
+--- TruffleHog will catch this (structural secret)
+PRIVATE_KEY = "-----BEGIN RSA PRIVATE KEY-----
+MIIEpQIBAAKCAQEA8P2m75Z1AM7QyUx48M9D1JKY%01
+wl12y2x4A5B6c7D8E9fG0h1I2j34K5L6M7N8O9PQ01023T4U5V6W7X8Y9Z0a1b2
+~3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8v9w0x1y2z3u4v5w6x7y8z9A0B1C2D3E4F5G6H7I8J9K0L1P2Q3R4S5T6U7V8W9X0Y1Z2a3b4c5d6e7f8g9h0i1j2k3l4m5n6
+~7p8q9r0s1t2u3v4w5x6y7z8A9B8C1D2E3F4G5H6I7J8K9L9M1N2O3P4Q5R6S7T8
+-----END RSA PRIVATE KEY-----"
+
+if __name__ == "__main__":
+    run_application()
+
+\\ No newline at end of file

=====
Reason: RSA private key
Date: 2026-02-21 00:18:53
Hash: 242f27748342eca38f70b74718293343463cfa98
Filepath: app/app.py
Branch: origin/main
Commit: Testing stripe secret block for presentation

-----BEGIN RSA PRIVATE KEY-----
=====
Reason: High Entropy
Date: 2026-02-21 00:16:30
Hash: 55253256e0e5dc2cd5d0857dd0ce0838b46476b
Filepath: app/app.py
Branch: origin/main
Commit: Testing secret block for presentation

@@ -1,18 +1,2 @@
-# A simple, secure script to test secret detection
```

```
-def run_application():
-    print('Application is starting up securely...')
-
---- STRUCTURAL SECRET (GUARANTEED BLOCK FOR SCREENSHOT) ---
-
---- Sengrep will ignore this (not a code vulnerability)
---- TruffleHog will catch this (structural secret)
-PRIVATE_KEY = "-----BEGIN RSA PRIVATE KEY-----
-MIIEpQIBAAKCAQEA8P2m75Z1AM7QyUx48M9D1JKY%01
-wl12y2x4A5B6c7D8E9fG0h1I2j34K5L6M7N8O9PQ01023T4U5V6W7X8Y9Z0a1b2
~3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8v9w0x1y2z3u4v5w6x7y8z9A0B1C2D3E4F5G6H7I8J9K0L1P2Q3R4S5T6U7V8W9X0Y1Z2a3b4c5d6e7f8g9h0i1j2k3l4m5n6
~7p8q9r0s1t2u3v4w5x6y7z8A9B8C1D2E3F4G5H6I7J8K9L9M1N2O3P4Q5R6S7T8
-----END RSA PRIVATE KEY-----"
-
-if __name__ == "__main__":
-    run_application()
-
-\\ No newline at end of file
+API_KEY = "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"
+\\ No newline at end of file

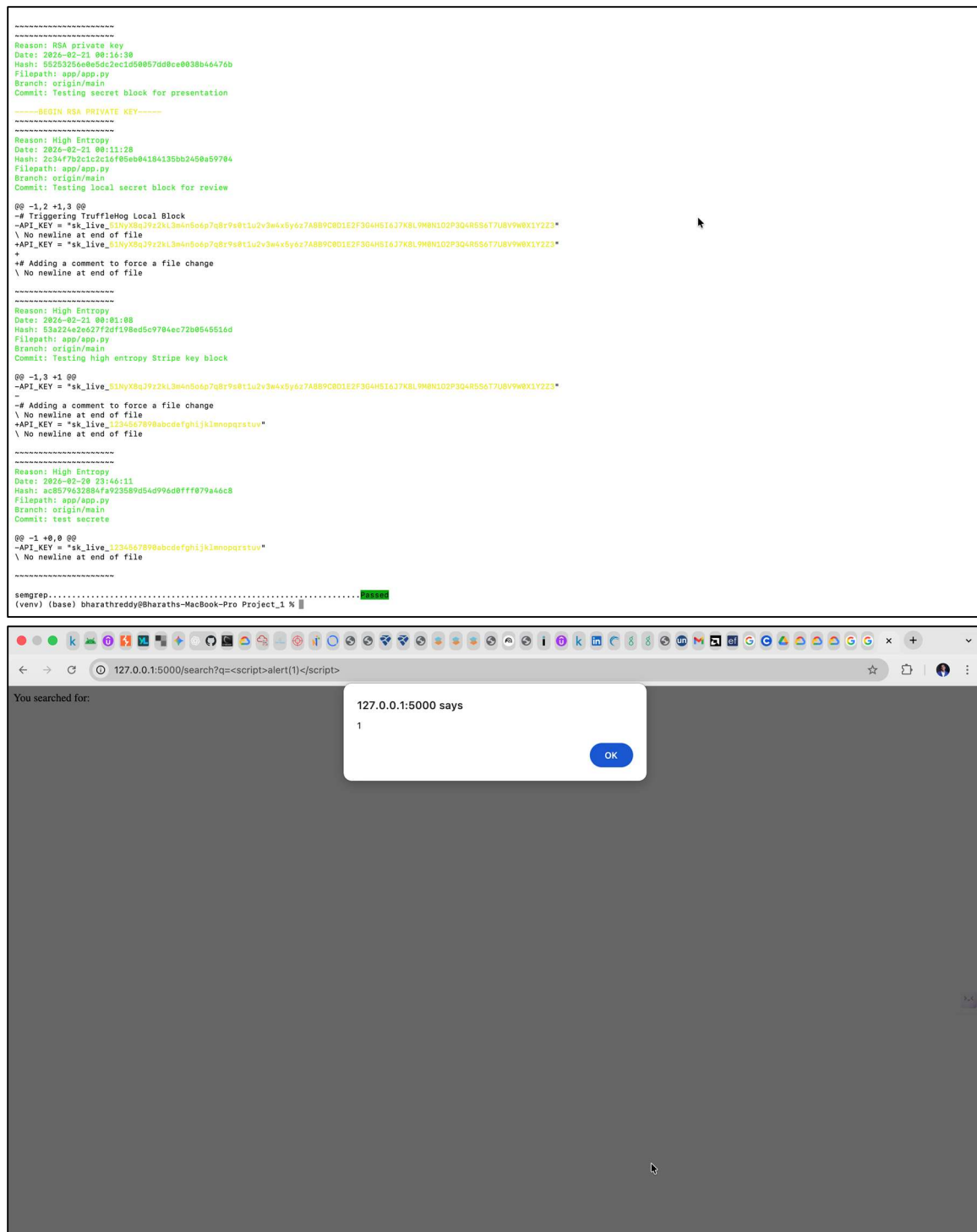
=====
Reason: RSA private key
Date: 2026-02-21 00:16:30
Hash: 55253256e0e5dc2cd5d0857dd0ce0838b46476b
Filepath: app/app.py
Branch: origin/main
Commit: Testing secret block for presentation

-----BEGIN RSA PRIVATE KEY-----
=====
Reason: High Entropy
Date: 2026-02-21 00:11:28
Hash: 2c34f7b2c1c2c16f08eb04184135bb2458a59784
Filepath: app/app.py
Branch: origin/main
Commit: Testing local secret block for review

@@ -1,2 +1,3 @@
-# Triggering TruffleHog Local Block
+
+API_KEY = "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"
+\\ No newline at end of file
+
+--- Adding a comment to force a file change
+\\ No newline at end of file

=====
Reason: High Entropy
Date: 2026-02-21 00:01:08
Hash: 55a24a2ae27726f198ed5c978aec72b054551ad
Filepath: app/app.py
Branch: origin/main
Commit: Testing high entropy Stripe key block

@@ -1,3 +1 @@
-API_KEY = "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"
-
-# Adding a comment to force a file change
+\\ No newline at end of file
```



5. Week 2 – SAST Integration (Semgrep)

Semgrep was integrated to detect insecure coding practices such as XSS vulnerabilities.

Commits containing vulnerable code are rejected locally and during GitHub CI pipeline execution.

```
name: DevSecOps Security Pipeline

on:
  push:
    branches: [ "master", "main" ]
  pull_request:
    branches: [ "master", "main" ]

jobs:
  security-scan:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0 # Required for history

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install Semgrep
        run: pip install semgrep
```

```
# --- TRUFFLEHOG FIX ---
# Removed 'base' and 'head' arguments.
# The action will now automatically detect the correct range.
- name: TruffleHog Secret Scan
  uses: trufflesecurity/trufflehog@main
  with:
    extra_args: --debug --only-verified

- name: Run SAST Scan
  run: semgrep --config=p/python .
```

After fixing the vulnerabilities, commits pass successfully.

```
branch: origin/main
Commit: Testing high entropy Strips key block

@@ -1,3 +1 @@
-API_KEY = "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"

-# Adding a comment to force a file change
\ No newline at end of file
+API_KEY = "sk_live_1234567890abcde7ghijklmnopqrstuv"
\ No newline at end of file

-----
Reason: High Entropy
Date: 2020-02-08 23:46:11
Hash: ac8679632884fa923589d5ad996d0fff879a4dc8
Filepath: app/app.py
Branch: origin/main
Commit: test secrets

@@ -1 +0,0 @@
-API_KEY = "sk_live_1234567890abcde7ghijklmnopqrstuv"
\ No newline at end of file

-----
semgrep..... Passed
(venv) (base) bharathreddy@Bharaths-MacBook-Pro Project_1 % rm -rf .git
(venv) (base) bharathreddy@Bharaths-MacBook-Pro Project_1 % git init
git add .
git commit -m "Clean secure baseline"
Initialized empty Git repository in /Users/bharathreddy/Downloads/Project_1/.git/
[main (root-commit) 88d93e7] Clean secure baseline
Committer: Bharath Reddy <bharathreddy@Bharaths-MacBook-Pro.local>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly:

    git config --global user.name "Your Name"
    git config --global user.email you@example.com

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

9 files changed, 133 insertions(+)
create mode 100644 .env.example
create mode 100644 .github/workflows/README.md
create mode 100644 .github/workflows/security.yml
create mode 100644 .gitignore
create mode 100644 .pre-commit-config.yaml
create mode 100644 app/app.py
create mode 100644 app/requirements.txt
create mode 100644 app/templates/index.html
create mode 100644 docs/sast.blocked.png
(venv) (base) bharathreddy@Bharaths-MacBook-Pro Project_1 % pre-commit run --all-files
trufflehog..... Passed
semgrep..... Passed
(venv) (base) bharathreddy@Bharaths-MacBook-Pro Project_1 % nano app/app.py
(venv) (base) bharathreddy@Bharaths-MacBook-Pro Project_1 % pre-commit run --all-files
trufflehog..... Passed
semgrep..... Passed
(venv) (base) bharathreddy@Bharaths-MacBook-Pro Project_1 %
```

6. Security Benefits

- Prevents credential leakage
- Prevents vulnerable code deployment
- Automated security enforcement
- Early vulnerability detection

7. Conclusion

The first two weeks successfully implemented local and CI-based security gates. This demonstrates real-world DevSecOps workflow where insecure code is blocked before production.

Week 3

Dynamic Analysis (DAST)

Dynamic Application Security Testing (DAST) tools perform runtime analysis of applications to identify vulnerabilities while the application is actively running. Unlike static analysis, which examines source code without execution, DAST operates in a black-box testing manner, simulating external attacks without requiring access to the application's internal code.

Week 3 Architecture:

Flask App → Docker Container → Running on localhost



OWASP ZAP Scan



Vulnerability Report

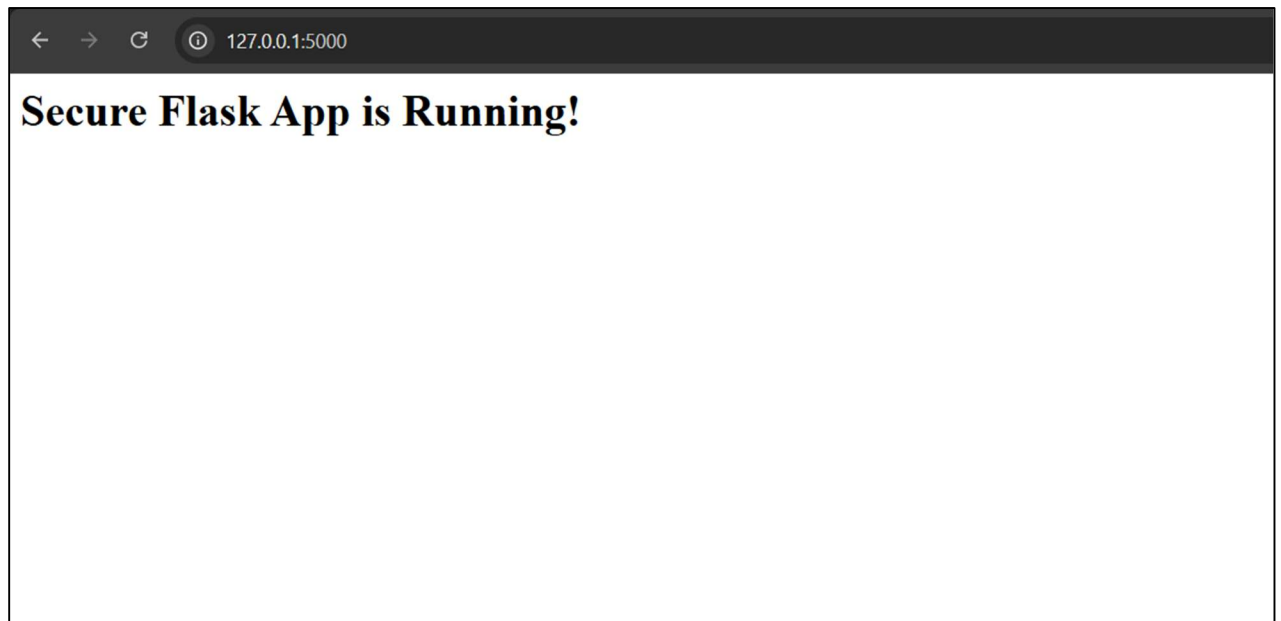
Building the Docker Image

```
● (venv) PS C:\Users\Pranav\OneDrive\Desktop\Project_1> docker build -t secure-flask-app .
[+] Building 19.0s (12/12) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 823B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 1.1s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [1/7] FROM docker.io/library/python:3.11-slim@sha256:fbaf3b73795df99960f4269b297420bde01a8631fc31ea3f121f2486d332d0 12.5s
=> => resolve docker.io/library/python:3.11-slim@sha256:fbaf3b73795df99960f4269b297420bde01a8631fc31ea3f121f2486d332d0 0.0s
=> => sha256:13159fd0b0512a3ecfe5d5e51affb0ef7eb36b371459c75e34f5c090a0870f4 1.29MB / 1.29MB 1.2s
=> => sha256:269d3f7471e27a9c2542916a49849e76630f22709b7e6063730b617d34d44d6f 14.36MB / 14.36MB 6.0s
=> => sha256:28c7e2bc4784ae35d32ed16d30b72e35df3c3f6a0214492f5be2b11e4b5ae2b0 250B / 250B 1.7s
=> => sha256:206356c42440674ecbdf1070cf70ce8ef7885ac2e5c56f1ecf800b758f6b0419 29.78MB / 29.78MB 11.1s
=> => extracting sha256:206356c42440674ecbdf1070cf70ce8ef7885ac2e5c56f1ecf800b758f6b0419 0.7s
=> => extracting sha256:13159fd0b0512a3ecfe5d5e51affb0ef7eb36b371459c75e34f5c090a0870f4 0.1s
=> => extracting sha256:269d3f7471e27a9c2542916a49849e76630f22709b7e6063730b617d34d44d6f 0.4s
=> => extracting sha256:28c7e2bc4784ae35d32ed16d30b72e35df3c3f6a0214492f5be2b11e4b5ae2b0 0.0s
=> [5/7] COPY app/ ./app/                                          0.1s
=> [6/7] RUN pip install --no-cache-dir -r requirements.txt       3.2s
=> [7/7] RUN chown -R appuser:appgroup /app                       0.3s
=> => exporting image                                              0.8s
=> => exporting layers                                             0.4s
=> => exporting manifest sha256:59532b6523768c2f53d492f67d053f25d670abdf917d2bd9484ce4e26a34268c 0.0s
=> => exporting config sha256:c96e1f050b8dbad6b4508225a19b26387da7b5ecce70a871c9ad149247759f91 0.0s
=> => exporting attestation manifest sha256:2962f7f51a53f263355f1b8b4b10bd263e73572d2663d2d0f3e6f6f5da057c73 0.0s
=> => exporting manifest list sha256:a8de984b3d316636141726f98f4d4916f18d9af45c021d1d274369213f637794 0.0s
=> => naming to docker.io/library/secure-flask-app:latest         0.0s
=> => unpacking to docker.io/library/secure-flask-app:latest      0.2s
(venv) PS C:\Users\Pranav\OneDrive\Desktop\Project_1>
```

Creating container for APP

```
(venv) PS C:\Users\Pranav\OneDrive\Desktop\Project_1> docker run -p 5000:5000 secure-flask-app
Application is starting up securely...
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [27/Feb/2026 14:20:34] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [27/Feb/2026 14:20:34] "GET /favicon.ico HTTP/1.1" 404 -
```

Now our app is accessible by localhost



Execution of OWASP ZAP Baseline Scan

```
Windows PowerShell
PS C:\Users\Pranav\OneDrive\Desktop\Project_1> docker run -v ${PWD}:/zap/wrk/:rw -t zaproxy/zap-stable zap-baseline.py -t http://host.docker.internal:5000 -r zap_report.html
Using the Automation Framework
Total of 3 URLs
PASS: Vulnerable JS Library (Powered by Retire.js) [10003]
PASS: In Page Banner Information Leak [10009]
PASS: Cookie No HttpOnly Flag [10010]
PASS: Cookie Without Secure Flag [10011]
PASS: Re-examine Cache-control Directives [10015]
PASS: Cross-Domain JavaScript Source File Inclusion [10017]
PASS: Content-Type Header Missing [10019]
PASS: Information Disclosure - Debug Error Messages [10023]
PASS: Information Disclosure - Sensitive Information in URL [10024]
PASS: Information Disclosure - Sensitive Information in HTTP Referrer Header [10025]
PASS: HTTP Parameter Override [10026]
PASS: Information Disclosure - Suspicious Comments [10027]
PASS: Off-site Redirect [10028]
PASS: Cookie Poisoning [10029]
PASS: User Controllable Charset [10030]
PASS: User Controllable HTML Element Attribute (Potential XSS) [10031]
PASS: Viewstate [10032]
PASS: Directory Browsing [10033]
PASS: Heartbleed OpenSSL Vulnerability (Indicative) [10034]
PASS: Strict-Transport-Security Header [10035]
PASS: Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s) [10037]
PASS: X-Backend-Server Header Information Leak [10039]
PASS: Secure Pages Include Mixed Content [10040]
```

The OWASP ZAP baseline scan was executed against the running Dockerized application using the ZAP CLI tool. The scan analyzes web pages for common vulnerabilities and security misconfigurations.

Initial OWASP ZAP Security Scan Report

**ZAP by Checkmarx**

ZAP Scanning Report

Site: <http://host.docker.internal:5000>

Generated on Fri, 27 Feb 2026 16:37:43

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	6
Informational	1
False Positives:	0

The initial ZAP scan report showing detected security issues in the application. The scan identified medium and low severity vulnerabilities related to missing security headers and information disclosure.

Detected Vulnerabilities During Active Scan

Summary of Sequences		
For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).		
Alerts		
Name	Risk Level	Number of Instances
Cross Site Scripting (DOM Based)	High	3
Cross Site Scripting (Reflected)	High	1
Source Code Disclosure - File Inclusion	High	1
Content Security Policy (CSP) Header Not Set	Medium	5
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	5
Storable and Cacheable Content	Informational	5

The ZAP active scan detected several high-severity vulnerabilities including Reflected Cross-Site Scripting (XSS) and DOM-based XSS, confirming that the scanner successfully identified intentionally introduced security flaws.

Implementation of HTTP Security Headers in Application

```
9  # ----- SECURITY HEADERS ----- #
10 @app.after_request
11 def set_security_headers(response):
12     response.headers['Content-Security-Policy'] = "default-src 'self';"
13     response.headers['X-Frame-Options'] = "DENY"
14     response.headers['X-Content-Type-Options'] = "nosniff"
15     response.headers['Referrer-Policy'] = "no-referrer"
16     response.headers['Permissions-Policy'] = "geolocation=(), microphone=()"
17     response.headers['Cross-Origin-Opener-Policy'] = "same-origin"
18     response.headers['Cross-Origin-Resource-Policy'] = "same-origin"
19     response.headers['Cross-Origin-Embedder-Policy'] = "require-corp"
20     return response
21 # ----- #
```

Security headers such as Content Security Policy, X-Frame-Options, and X-Content-Type-Options were implemented in the Flask application to mitigate web security vulnerabilities detected during the initial scan.

ZAP Scan Results After Vulnerability Remediation



ZAP Scanning Report

Site: http://host.docker.internal:5000

Generated on Fri, 27 Feb 2026 17:02:59

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	1
Low	1
Informational	1
False Positives:	0

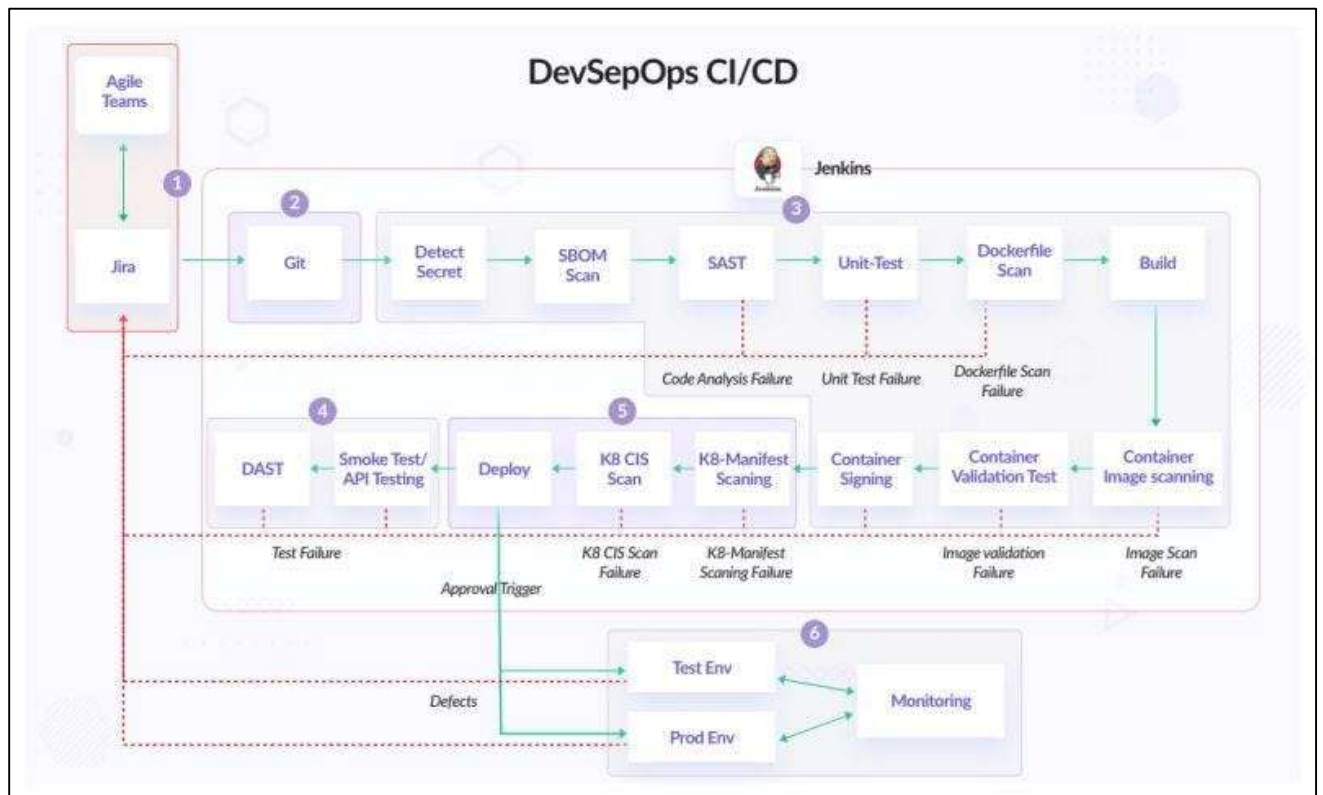
After implementing security controls, the application was rescanned using OWASP ZAP. The results show a significant reduction in detected vulnerabilities, demonstrating successful mitigation of previously identified issues.

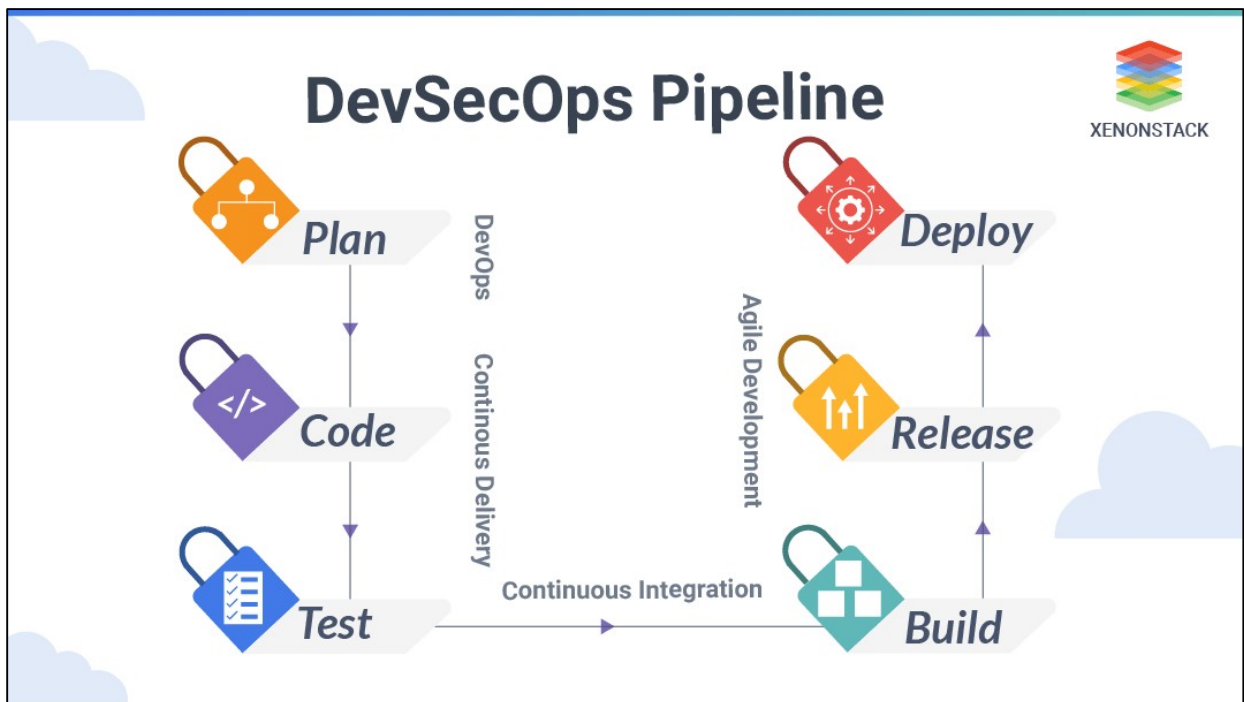
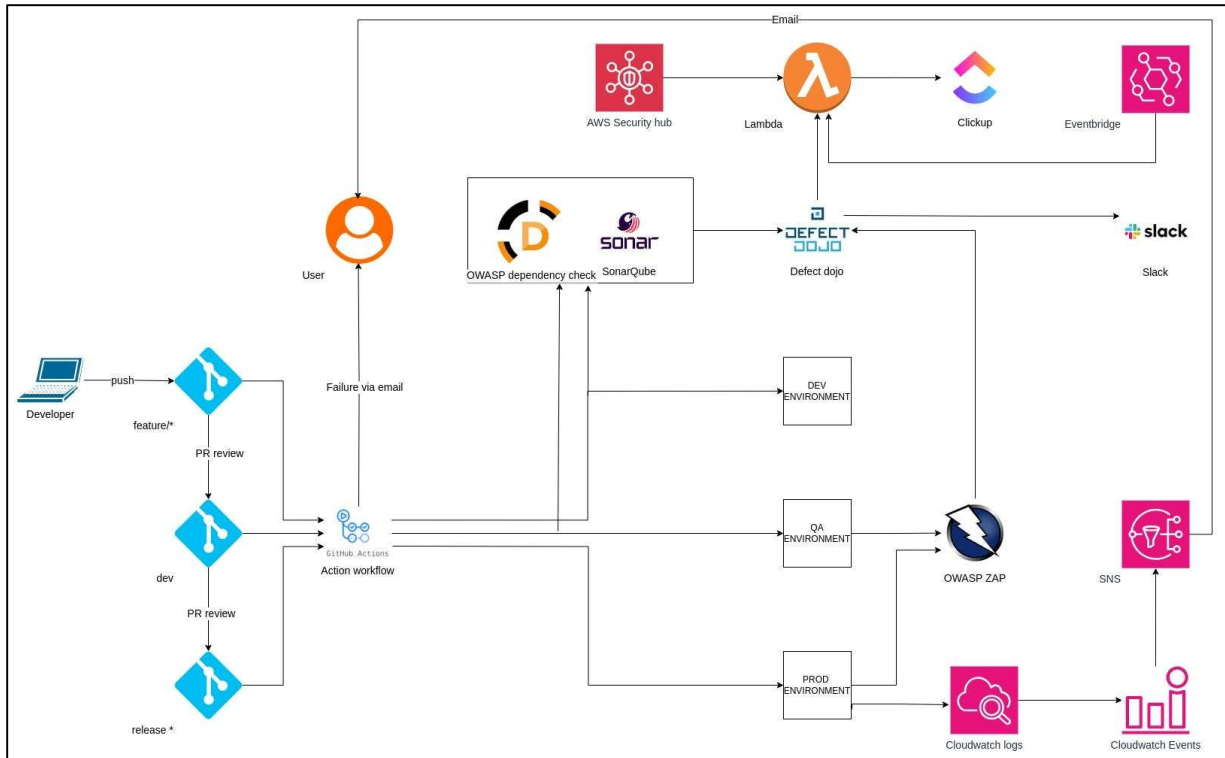
Week 4

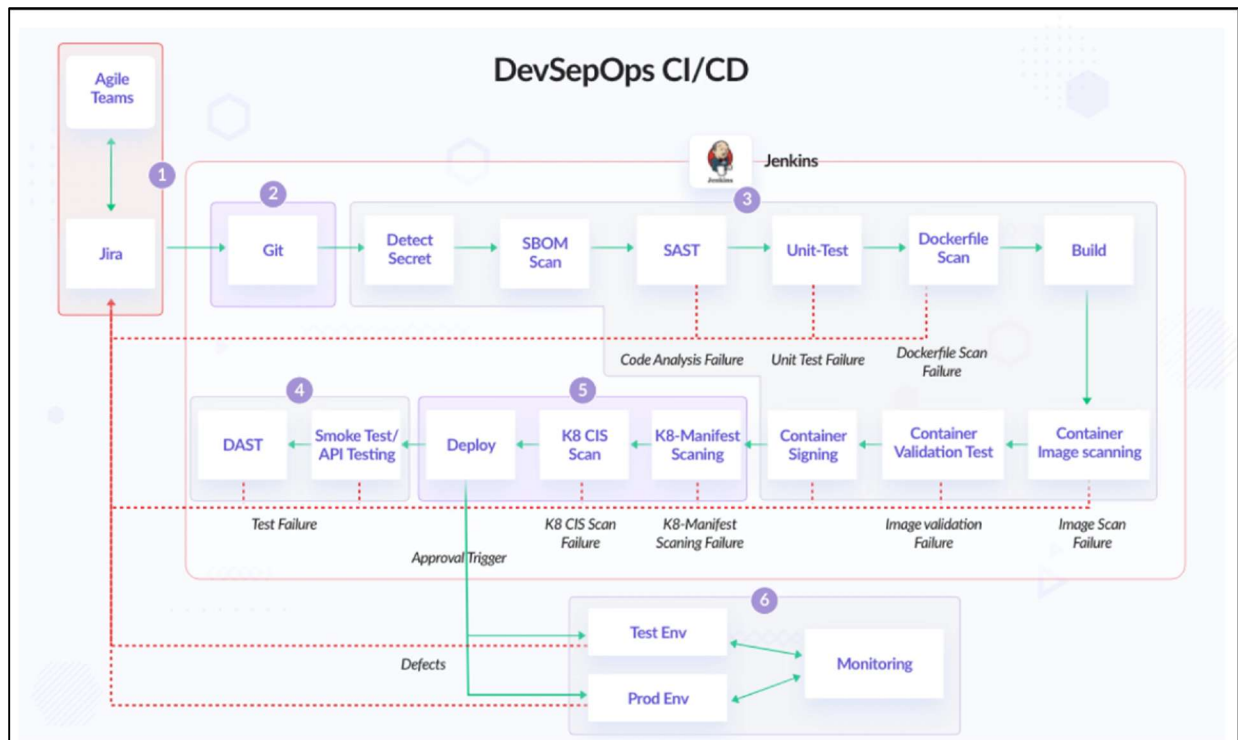
DevSecOps CI/CD Security Automation

The objective of Week 4 was to automate security testing using a CI/CD pipeline. GitHub Actions was used to integrate secret scanning, static application security testing (SAST), container build, and dynamic application security testing (DAST). The pipeline ensures that security checks run automatically whenever new code is pushed to the repository.

CI/CD Pipeline Architecture







The DevSecOps pipeline integrates security checks into the CI/CD workflow. Each time a developer pushes code to the repository, automated security tools run sequentially to detect vulnerabilities before deployment.

GitHub Actions Workflow Setup

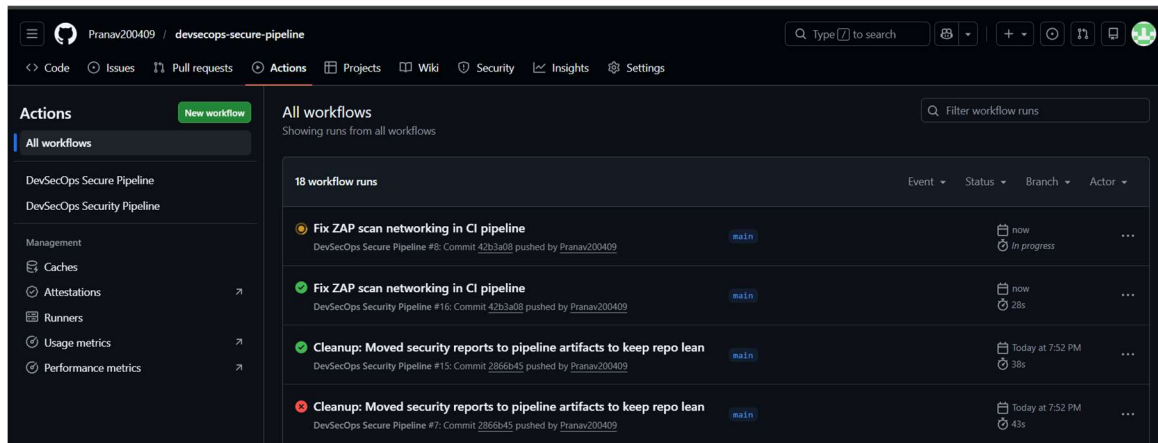
GitHub Actions provides an automated CI/CD environment that runs workflows based on repository events such as code pushes or pull requests.

workflow file location: `.github/workflows/devsecops-pipeline.yml`

GitHub repository

The screenshot shows the GitHub repository page for 'Pranav200409/devsecops-secure-pipeline'. The repository is public and has 20 commits. The file browser shows a directory structure with files like `.github/workflows`, `app/templates`, `docs`, `env.example`, `.gitignore`, `Dockerfile`, `README.md`, `requirements.txt`, and `zap.yml`. The README file is selected, showing the project's purpose: 'A hands-on DevSecOps project demonstrating shift-left security and automated CI security scanning'. It lists security features implemented, including Local Developer Security (Pre-commit secret detection, SAST, Commit blocking) and CI/CD Security (Automatic secret scanning, Static Application Security Testing, Pipeline fails on vulnerabilities). The architecture is described as 'Developer -> Pre-commit Hooks -> GitHub -> GitHub Actions Security Scan -> Secure Code Only'. Vulnerabilities prevented include Hardcoded API keys, XSS vulnerabilities, and Unsafe Flask configurations. Tools used are Python Flask, TruffleHog, Semgrep, and GitHub Actions. The 'How to Run' section provides a code snippet: `pip install -r requirements.txt` and `flask --app app run`. The project goal is to 'Simulate real-world DevSecOps workflow where insecure code never reaches production repositories.'

.github/workflows folder



Secret Scanning Stage (TruffleHog)

TruffleHog is used to detect hardcoded secrets such as API keys, tokens, and passwords in the repository.

```
- name: Run TruffleHog Secret Scan
  uses: trufflesecurity/trufflehog@main
  with:
    path: ./
```

Static Application Security Testing (SAST)

Static code analysis was performed using Semgrep to detect potential security vulnerabilities in the source code. Semgrep scans the application source code for vulnerabilities related to the OWASP Top 10 security risks.

```
- name: Run Semgrep SAST
  uses: returntocorp/semgrep-action@v1
  with:
    config: "p/owasp-top-ten"
```

Docker Image Build

The application was containerized using Docker to ensure consistent runtime environments across development and testing stages.

```
- name: Build Docker Image
  run: docker build -t secure-flask-app .
```

Application Deployment for Testing

After building the container image, the application container is started inside the CI environment so that dynamic security testing can be performed.

```
- name: Run Container
  run: docker run -d -p 5000:5000 secure-flask-app
```

Week 4 Conclusion

Week 4 successfully implemented a DevSecOps CI/CD pipeline that automates security testing during the development lifecycle. The pipeline integrates secret scanning, static code analysis, container build, and dynamic application security testing. Automated security checks help detect vulnerabilities early and improve the overall security posture of the application.

Overall Project Conclusion

This project demonstrates the implementation of a secure DevSecOps pipeline for a containerized web application. The objective was to integrate security practices throughout the software development lifecycle by combining automated security tools with CI/CD workflows.

The application was developed using Flask and containerized using Docker to ensure consistent deployment environments. Multiple security layers were integrated into the pipeline, including secret scanning, static application security testing (SAST), and dynamic application security testing (DAST). Tools such as TruffleHog and Semgrep were used to identify exposed secrets and vulnerable code patterns, while OWASP ZAP was used to perform runtime vulnerability scanning on the deployed application.

During testing, intentional vulnerabilities were introduced to verify that the security tools could successfully detect issues such as cross-site scripting (XSS). These vulnerabilities were then remediated, demonstrating how automated security checks can help identify and mitigate risks early in the development process.

The pipeline was automated using GitHub Actions, ensuring that every code push triggered security checks, container builds, and vulnerability scans. This approach helps maintain continuous security monitoring and prevents insecure code from progressing through the deployment process.

Overall, the project highlights the importance of integrating security into the DevOps workflow. By adopting DevSecOps practices, organizations can improve software quality, detect vulnerabilities earlier, and maintain a stronger security posture throughout the development lifecycle.

Key Achievements

Project Achievements

- Implemented a secure CI/CD pipeline using GitHub Actions
- Integrated Secret Scanning (TruffleHog)
- Performed Static Security Analysis (Semgrep)
- Implemented Dynamic Security Testing (OWASP ZAP)
- Containerized application using Docker
- Demonstrated vulnerability detection and remediation
- Automated security checks for every code commit

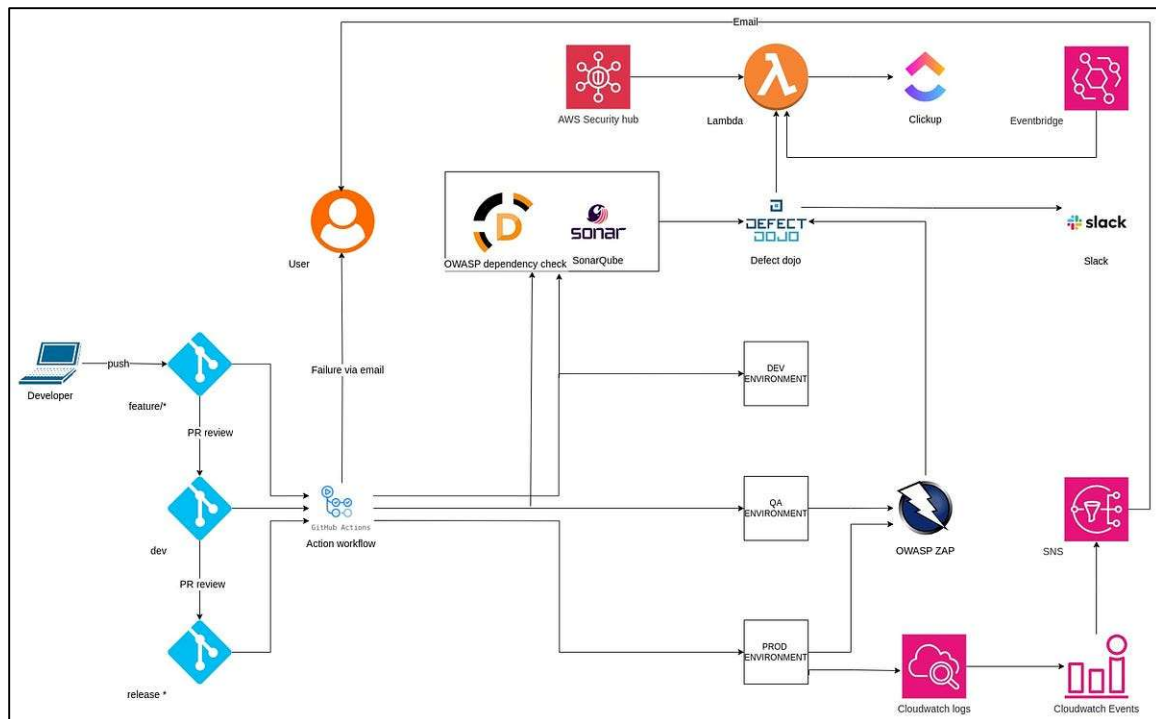


Figure 1:Architecture of the implemented DevSecOps secure pipeline integrating secret scanning, SAST, containerization, and DAST testing.